



UNIVERZITET U SARAJEVU
ELEKTROTEHNIČKI FAKULTET
ODSJEK ZA AUTOMATIKU I ELEKTRONIKU

Laboratorijska vježba 2

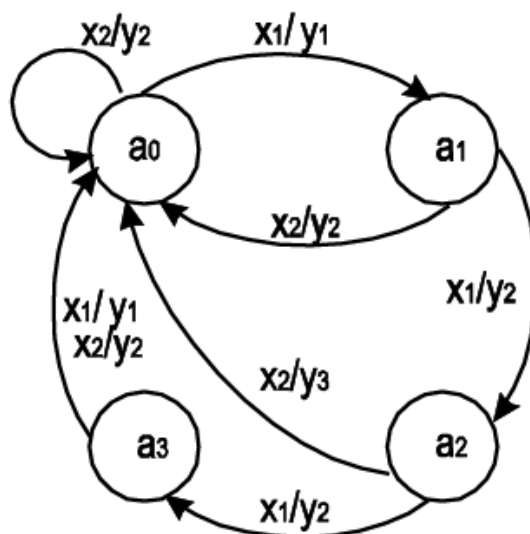
Praktikum iz logičkog dizajna i upravljanja zasnovan na FPGA razvojnom sistemu Gowin 9k

Tarik Ćatić

U okviru ove laboratorijske vježbe implementiraju se dva konačna automata – Mealyev i Mooreov. Za oba automata izvodi se tablica prelaza i izlaza na osnovu zadanog grafa, provjerava se mogućnost minimizacije broja unutrašnjih stanja, vrši se kodiranje stanja i signala, te se automat implementira na dva načina: korištenjem isključivo tablice prelaza i izlaza (bihevioralni opis), te korištenjem strukturne sheme automata zasnovane na T flip-flopovima.

Zadatak 1 – Mealyev automat

Graf koji opisuje funkcionalnost automata prikazan je na slici 1.



Slika 1: Graf Mealyevog automata (zadatak 1).

Automat ima četiri unutrašnja stanja (a_0, a_1, a_2, a_3), dva ulazna signala (x_1, x_2) i tri moguća izlazna signala (y_1, y_2, y_3). Na osnovu grafa se dobija tablica prelaza i izlaza, data u tabeli 1.

Tabela 1: Tablica prelaza i izlaza (zadatak 1).

$x_n a_r$	a_0	a_1	a_2	a_3
x_1	a_1/y_1	a_2/y_2	a_3/y_2	a_0/y_1
x_2	a_0/y_2	a_0/y_2	a_0/y_3	a_0/y_2

Provjera minimizacije broja stanja

Prije prelaska na strukturnu sintezu, potrebno je provjeriti postoje li ekvivalentna stanja koja bi se mogla objediniti. Stanja a_0 i a_3 imaju identičan izlazni par (y_1, y_2) za ulaze (x_1, x_2), pa predstavljaju kandidate za spajanje. Međutim, da bi $a_0 \equiv a_3$ vrijedilo, potrebno je da njihovi nasljednici po istom ulazu takođe budu ekvivalentni – za x_1 , a_0 prelazi u a_1 , a a_3 prelazi u a_0 , pa je uslov ekvivalencije $a_1 \equiv a_0$. Kako a_1 ima izlazni par (y_2, y_2), različit od izlaznog para stanja a_0 , uslov $a_1 \equiv a_0$ nije ispunjen, pa a_0 i a_3 nisu ekvivalentna stanja. Provjerom preostalih parova stanja utvrđuje se da automat ne sadrži ekvivalentna stanja, te je već zadan u minimalnom obliku sa četiri stanja.

Kodiranje signala

Pošto automat ima četiri stanja, za realizaciju registra stanja potrebna su dva T flip-flopa (Q_1, Q_0). Kodiranje stanja, ulaza i izlaza dato je u tabelama 2–4.

Tabela 2: Kodirana stanja.

	Q_0	Q_1
a_0	0	0
a_1	0	1
a_2	1	0
a_3	1	1

Tabela 3: Kodirani ulaz.

	U
x_1	0
x_2	1

Tabela 4: Kodirani izlazi.

	Z_1	Z_2
y_1	0	1
y_2	1	0
y_3	1	1

Prošireno kodirana tablica i logičke funkcije

Na osnovu kodiranih tablica formira se proširena kodirana tablica prelaza i izlaza, data u tabeli 5, gdje se za svaku kombinaciju $U(t)$, $Q_0(t)$ i $Q_1(t)$ određuje sljedeće stanje $Q_0(t+1)$, $Q_1(t+1)$, potrebni pobudni signali T flip-floпова ($T_1 = Q_1(t) \oplus Q_1(t+1)$, $T_2 = Q_0(t) \oplus Q_0(t+1)$), te kodirani izlazi Z_1, Z_2 .

Tabela 5: Proširena kodirana tablica (zadatak 1).

U	$Q_0(t)$	$Q_1(t)$	$Q_0(t+1)$	$Q_1(t+1)$	T_1	T_2	Z_1	Z_2
0	0	0	1	0	0	1	0	1
0	0	1	1	1	0	1	1	0
0	1	0	0	1	1	1	1	0
0	1	1	0	0	1	1	0	1
1	0	0	0	0	0	0	1	0
1	0	1	0	0	1	0	1	1
1	1	0	0	0	0	1	1	0
1	1	1	0	0	1	1	1	0

Minimizacijom logičkih funkcija iz proširene tablice (metodom Veitchovih dijagrama) dobijaju se sljedeće optimalne forme:

$$T_1 = Q_1U \vee Q_0\bar{U} \quad (1)$$

$$T_2 = Q_0 \vee \bar{U} \quad (2)$$

$$Z_1 = U \vee (Q_1 \oplus Q_0) \quad (3)$$

$$Z_2 = Q_1Q_0\bar{U} \vee Q_1U\bar{Q}_0 \vee \bar{Q}_1\bar{Q}_0\bar{U} \quad (4)$$

Na osnovu ovih jednačina formira se strukturna shema automata, sastavljena od dva T flip-flopa i osnovnih logičkih kola (NE, I, ILI).

SystemVerilog implementacija

Implementacija korištenjem isključivo tablice prelaza i izlaza data je u Kodu 1:

Kod 1 Implementacija Mealyevog automata korištenjem tablice prelaza i izlaza.

```
1 module mealy_automat (
2     input logic      U,
3     input logic      Cp,
4     input logic      reset,
5     output logic [1:0] Z,
6     output logic [1:0] stanje
7 );
8
9     logic [1:0] sljed_stanje;
10
11     always_ff @(posedge reset or posedge Cp) begin
12         if (reset == 1'b1) begin
13             stanje <= 2'd0;
14         end
15         else begin
16             stanje <= sljed_stanje;
17         end
18     end
19
```

```

20 always_comb begin
21     case (stanje)
22         2'd0: begin
23             if (U == 1'b0) begin
24                 Z = 2'b01;
25                 sljed_stanje = 2'd1;
26             end
27             else begin
28                 Z = 2'b10;
29                 sljed_stanje = 2'd0;
30             end
31         end
32         2'd1: begin
33             if (U == 1'b0) begin
34                 Z = 2'b10;
35                 sljed_stanje = 2'd2;
36             end
37             else begin
38                 Z = 2'b10;
39                 sljed_stanje = 2'd0;
40             end
41         end
42         2'd2: begin
43             if (U == 1'b0) begin
44                 Z = 2'b10;
45                 sljed_stanje = 2'd3;
46             end
47             else begin
48                 Z = 2'b11;
49                 sljed_stanje = 2'd0;
50             end
51         end
52         2'd3: begin
53             if (U == 1'b0) begin
54                 Z = 2'b01;
55                 sljed_stanje = 2'd0;
56             end
57             else begin
58                 Z = 2'b10;           // y2
59                 sljed_stanje = 2'd0; // a0
60             end
61         end
62         default: begin
63             Z = 2'b00;
64             sljed_stanje = 2'd0;
65         end
66     endcase
67 end
68
69 endmodule : mealy_automat

```

Implementacija korištenjem strukturne sheme automata, zasnovana na izvedenim jednačima T_1 , T_2 , Z_1 i Z_2 , data je u Kodu 2. Modul `T_ff` opisuje pojedinačni T flip-flop, dok modul `mealy_struktura` povezuje dva flip-flopa sa kombinacionom logikom sljedećeg stanja i izlaza.

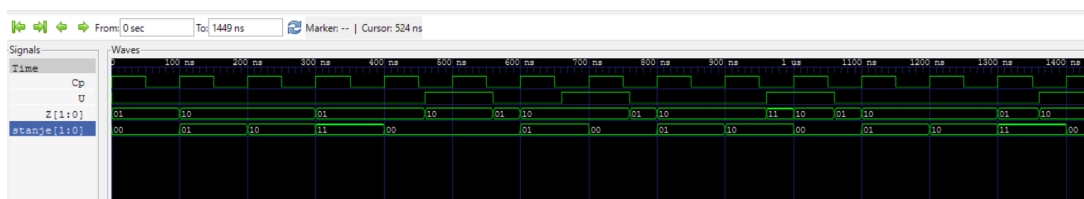
Kod 2 Implementacija Mealyevog automata korištenjem strukturne sheme (T flip-flopi).

```
1 module T_ff (
2     input logic T,
3     input logic Cp,
4     input logic reset,
5     output logic Q
6 );
7
8     always_ff @(posedge reset or posedge Cp) begin
9         if (reset == 1'b1) begin
10             Q <= 1'b0;
11         end
12         else begin
13             Q <= T ^ Q;
14         end
15     end
16
17 endmodule : T_ff
18
19
20 module mealy_struktura (
21     input logic U,
22     input logic Cp,
23     input logic reset,
24     output logic [1:0] Z,
25     output logic [1:0] stanje
26 );
27
28     logic Q1, Q0;
29     logic T1, T2;
30
31     T_ff flipflop1 (.T(T1), .Cp(Cp), .reset(reset), .Q(Q1));
32     T_ff flipflop2 (.T(T2), .Cp(Cp), .reset(reset), .Q(Q0));
33
34     always_comb begin
35         T1 = (Q1 & U) | (Q0 & ~U);
36         T2 = Q0 | ~U;
37
38         Z[1] = U | (Q1 ^ Q0);
39         Z[0] = (Q1 & Q0 & ~U) | (Q1 & U & ~Q0) | (~Q1 & ~Q0 & ~U);
40
41         stanje = {Q1, Q0};
42     end
43
44 endmodule : mealy_struktura
```

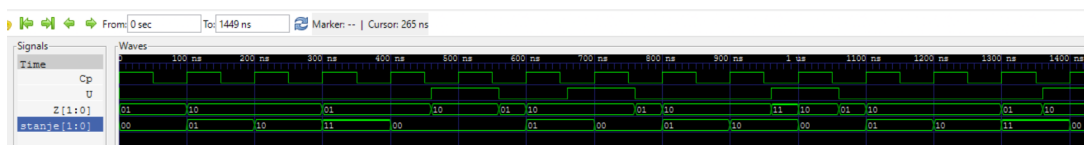
Simulacija putem Verilator alata

Fajlovi potrebni za simulaciju ovih primjera se mogu pronaći na github linku.

Prikaz ulaznih i izlaznih signala dobijenih simulacijom koda koji koristi tablicu prelaza i izlaza dat je na slici 2, a koristeći strukturnu shemu se nalazi na slici 3. Poređenjem dobijenih talasnih oblika uočava se da su nizovi vrijednosti signala `stanje` i `Z` potpuno identični kod oba pristupa, za istu sekvencu ulaznih vrijednosti `U` – čime je potvrđeno da je strukturna implementacija, zasnovana na izvedenim jednačinama T_1 , T_2 , Z_1 i Z_2 , funkcionalno ekvivalentna bihevioralnoj implementaciji zasnovanoj isključivo na tablici prelaza i izlaza.



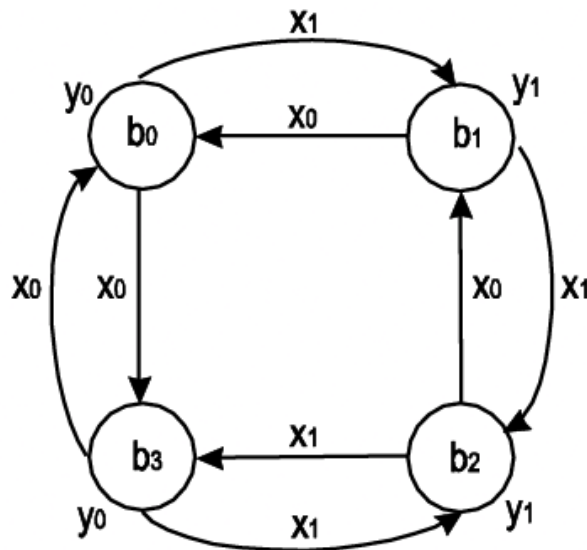
Slika 2: Ulazni i izlazni signali Mealyevog automata koristeći tablicu prelaza i izlaza.



Slika 3: Ulazni i izlazni signali Mealyevog automata koristeći strukturnu shemu.

Zadatak 2 – Mooreov automat

Graf koji opisuje funkcionalnost automata prikazan je na slici 4.



Slika 4: Graf Mooreovog automata (zadatak 2).

Automat ima četiri unutrašnja stanja (b_0, b_1, b_2, b_3), sa dva ulazna signala (x_0, x_1) i dva moguća izlazna signala (y_0, y_1). Za razliku od Mealyevog automata, izlaz Mooreovog automata zavisi isključivo od trenutnog stanja, a ne i od trenutnog ulaza. Prelazi automata ne formiraju jednostavan simetričan ciklus – iz stanja b_0, b_1 i b_2 ulaz x_1 redom vodi u sljedeće stanje ($b_0 \rightarrow b_1 \rightarrow b_2 \rightarrow b_3$), dok iz stanja b_3 ulaz x_1 vraća automat u b_2 (ne dalje ka b_0). Slično, ulaz x_0 iz stanja b_1, b_2 i b_3 redom vraća automat unazad ($b_1 \rightarrow b_0, b_2 \rightarrow b_1, b_3 \rightarrow b_0$), dok iz stanja b_0 ulaz x_0 vodi u b_3 . Tablica prelaza i izlaza data je u tabeli 6.

Tabela 6: Tablica prelaza i izlaza (zadatak 2).

$x_n b_r$	b_0	b_1	b_2	b_3
x_0	b_3	b_0	b_1	b_0
x_1	b_1	b_2	b_3	b_2
y_i	y_0	y_1	y_1	y_0

Provjera minimizacije broja stanja

Stanja b_0 i b_3 dijele isti izlaz y_0 , dok b_1 i b_2 dijele isti izlaz y_1 , pa predstavljaju kandidate za spajanje. Za $b_0 \equiv b_3$ bilo bi potrebno da za ulaz x_1 njihovi nasljednici (b_1 za b_0 , b_2 za b_3) budu ekvivalentni, odnosno $b_1 \equiv b_2$. Provjerom tog para: za ulaz x_0 , nasljednici b_1 i b_2 su redom b_0 i b_1 , pa bi uslov $b_1 \equiv b_2$ zahtijevao da vrijedi $b_0 \equiv b_1$ – što ne vrijedi, jer b_0 i b_1 imaju različite izlaze (y_0 naspram y_1). Prema tome, $b_1 \not\equiv b_2$, pa ni $b_0 \not\equiv b_3$. Automat, dakle, nema ekvivalentnih stanja i već je zadan u minimalnom obliku.

Kodiranje signala

Kao i u prethodnom zadatku, za realizaciju registra stanja potrebna su dva T flip-flopa. Pošto Mooreov izlaz zavisi isključivo od stanja, dovoljan je jedan izlazni bit Z . Kodiranje je dato u tabelama 7–9.

Tabela 7: Kodirana stanja.

	Q_0	Q_1
b_0	0	0
b_1	1	0
b_2	0	1
b_3	1	1

Tabela 8: Kodirani ulaz.

	U
x_0	0
x_1	1

Tabela 9: Kodirani izlazi.

	Z
y_0	0
y_1	1

Prošireno kodirana tablica i logičke funkcije

Proširena kodirana tablica prelaza data je u tabeli 10.

Tabela 10: Proširena kodirana tablica (zadatak 2).

U	$Q_0(t)$	$Q_1(t)$	$Q_0(t+1)$	$Q_1(t+1)$	T_1	T_2
0	0	0	1	1	1	1
0	0	1	1	0	1	1
0	1	0	0	0	0	1
0	1	1	0	0	1	1
1	0	0	1	0	0	1
1	0	1	1	1	0	1
1	1	0	0	1	1	1
1	1	1	0	1	0	1

Uočava se da je $T_2 = 1$ za svaku kombinaciju ulaza – flip-flop Q_0 mijenja stanje pri svakom taktu, nezavisno od ulaza. Minimizacijom preostalih funkcija (metodom Veitchovih dijagrama, provjereno i računski) dobija se:

$$T_1 = Q_1 \bar{U} \vee \bar{Q}_0 \bar{U} \vee Q_0 U \bar{Q}_1 \quad (5)$$

$$T_2 = 1 \quad (6)$$

$$Z = Q_1 \oplus Q_0 \quad (7)$$

SystemVerilog implementacija

Implementacija korištenjem isključivo tablice prelaza i izlaza data je u Kodu 3:

Kod 3 Implementacija Mooreovog automata korištenjem tablice prelaza i izlaza.

```

1 module moore_automat (
2     input logic      U,
3     input logic      Cp,
4     input logic      reset,
5     output logic     Z,
6     output logic [1:0] stanje
7 );
8
9     logic [1:0] sljed_stanje;
10

```

```

11  always_ff @(posedge reset or posedge Cp) begin
12      if (reset == 1'b1) begin
13          stanje <= 2'd0;
14      end
15      else begin
16          stanje <= sljed_stanje;
17      end
18  end
19
20  always_comb begin
21      case (stanje)
22          2'd0: begin
23              Z = 1'b0;
24              sljed_stanje = (U == 1'b0) ? 2'd3 : 2'd1;
25          end
26          2'd1: begin
27              Z = 1'b1;
28              sljed_stanje = (U == 1'b0) ? 2'd0 : 2'd2;
29          end
30          2'd2: begin
31              Z = 1'b1;
32              sljed_stanje = (U == 1'b0) ? 2'd1 : 2'd3;
33          end
34          2'd3: begin
35              Z = 1'b0;
36              sljed_stanje = (U == 1'b0) ? 2'd0 : 2'd2;
37          end
38          default: begin
39              Z = 1'b0;
40              sljed_stanje = 2'd0;
41          end
42      endcase
43  end
44
45  endmodule : moore_automat

```

Implementacija korištenjem strukturne sheme automata, zasnovana na izvedenim jednačima T_1 , T_2 i Z , data je u Kodu 4. Iskorišten je isti modul `T_ff` definisan u zadatku 1.

Kod 4 Implementacija Mooreovog automata korištenjem strukturne sheme (T flip-flopovi).

```

1  module T_ff (
2      input  logic T,
3      input  logic Cp,
4      input  logic reset,
5      output logic Q
6  );
7
8      always_ff @(posedge reset or posedge Cp) begin
9          if (reset == 1'b1) begin

```

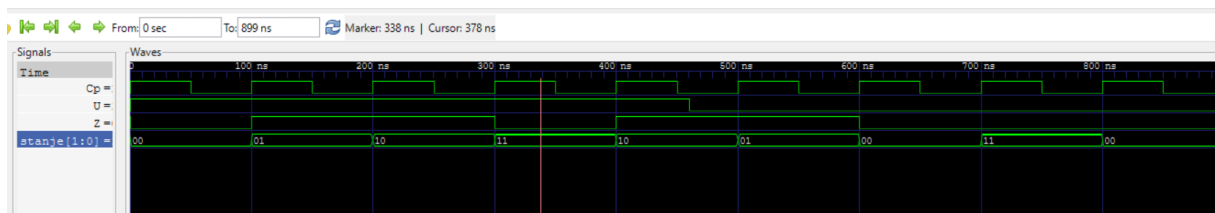
```

10         Q <= 1'b0;
11     end
12     else begin
13         Q <= T ^ Q;
14     end
15 end
16
17 endmodule : T_ff
18
19
20 module moore_struktura (
21     input logic      U,
22     input logic      Cp,
23     input logic      reset,
24     output logic      Z,
25     output logic [1:0] stanje // trenutno stanje automata (Q1,Q0
    ) - vidljivo u simulaciji
26 );
27
28     logic Q1, Q0;
29     logic T1, T2;
30
31     T_ff flipflop1 (.T(T1), .Cp(Cp), .reset(reset), .Q(Q1));
32     T_ff flipflop2 (.T(T2), .Cp(Cp), .reset(reset), .Q(Q0));
33
34     always_comb begin
35         T1 = (Q1 & ~U) | (~Q0 & ~U) | (Q0 & U & ~Q1);
36         T2 = 1'b1;
37
38         Z = Q1 ^ Q0;
39         stanje = {Q1, Q0};
40     end
41
42 endmodule : moore_struktura

```

Simulacija putem Verilator alata

Kod za fileove run-sim i versim se može naći na github repozitoriju. Prikaz ulaznih i izlaznih signala dobijenih simulacijom oba pristupa dat je na slici 5.



Slika 5: Ulazni i izlazni signali Mooreovog automata.